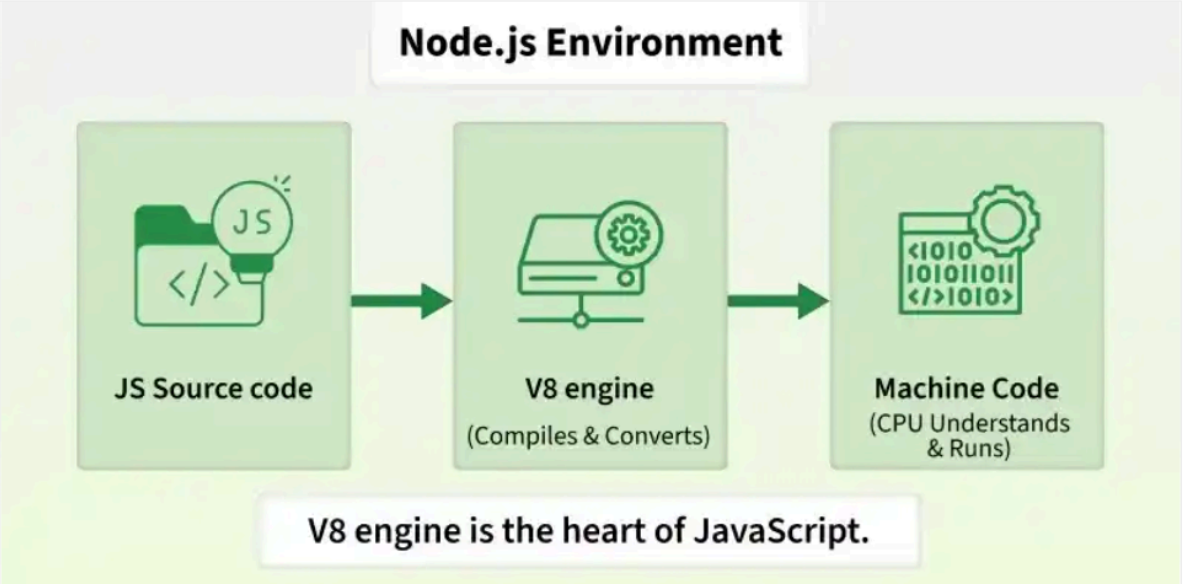
Introduction To Node JS -

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It enables developers to run JavaScript outside the browser to build fast, scalable server-side applications.

- JavaScript was initially a frontend-only language, and Node.js (2009) enabled backend development as well.
- Non-blocking, event-driven architecture for high performance.
- Supports the creation of REST APIs, real-time applications, and microservices.
- Comes with a rich library of modules through npm (Node Package Manager).

*NOTE - Node.js is **not a programming language** and it is **not a framework**. It is a runtime environment for executing JavaScript code without a browser.*



What is the V8 Engine?

The V8 engine is Google's open-source JavaScript engine, used by Chrome and Node.js. Without V8 engine, Node.js would not be able to directly execute JavaScript.

It compiles JavaScript to native machine code for fast execution.

- **Origin:** Developed by Google for Chrome in 2008
- **Integration:** Node.js uses V8 to provide a JavaScript runtime on the server
- **Features:** Just-In-Time compilation, ES6+ support

Creation of Node.js -

In **2009**, **Ryan Dahl** introduced Node.js.

His goal was to create a way to use JavaScript for **server-side programming** with an efficient, event-driven architecture.

Node.js used Google's **V8 JavaScript engine** as its JavaScript execution engine.

Installation of Node.js -

Go to official website - https://nodejs.org/en/download/current
Select the latest version and download the installer according to your operating system. Run the installer and keep the default options.

Verify the installation -

After installation, open:
Windows - Command Prompt / PowerShell on VS code
macOS/Linux - Terminal

Run: node -v

You should get something like:
v26.8.1

The exact version will depend on the current LTS release.

First Node.js program -

Create a folder and open it in VS Code.

Create app.js file.

Write: **console.log("Hello from Node.js!");**

Now open the terminal and run:

node app.js

Output:
Hello from Node.js!

That's your first Node program. Here, you will notice that javascript code ran without any browser because it ran with the help of Node.js. It provided the environment(V8 engine) to run the code.

Node provides its own environment and APIs.

For example, Node can interact with the filesystem:

```
const fs = require("fs");
fs.writeFileSync("hello.txt", "Hello Node!");
```

Run: node app.js

A file called: **hello.txt** will be created.

NOTE - JavaScript is now interacting with the computer rather than just the webpage.

Node.js is widely used for:

- Web servers
- REST APIs
- Real-time applications
- Microservices
- Command-line applications
- Backend applications
- Streaming applications

Features of Node.js -

1. JavaScript Runtime -

Node.js allows us to execute JavaScript outside the browser.

```
const name = "Ekta";
console.log(`Hello ${name}`);
```

Run: **node app.js**

2. Built on V8 Engine -

Node.js uses Google's **V8 JavaScript engine** to execute JavaScript.

V8 converts JavaScript into machine-level instructions efficiently.

3. Asynchronous -

Node.js supports **asynchronous operations**.

For example, when Node.js needs to read a file, it doesn't necessarily have to stop the entire application while waiting for the file operation to finish. The program can continue doing other work while the file operation is being completed.

4. Non-Blocking I/O -

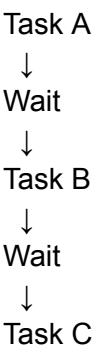
I/O = Input/Output

Examples:

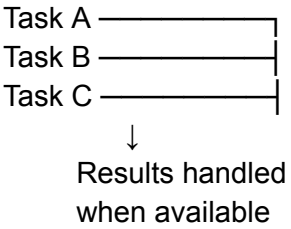
- Reading files
- Writing files
- Database operations
- Network requests

Node.js uses a non-blocking approach for many I/O operations.

Blocking approach



Non-blocking approach



This makes Node.js particularly useful for applications that handle many I/O operations.

5. Event-Driven -

Node.js follows an **event-driven architecture**.

An event can be something like:

- Request received
- File reading completed
- User connected
- Database operation completed

6. Single-Threaded JavaScript Execution -

Node.js executes JavaScript code using a **single main JavaScript thread**. However, this does **not** mean Node.js can only perform one operation at a time. Node.js can handle asynchronous I/O using its event loop and underlying system mechanisms.

7. Fast and Scalable -

Because of its event-driven and non-blocking architecture, Node.js can efficiently handle applications with many concurrent I/O operations.

It is commonly used for:

- Chat applications
- APIs
- Streaming applications
- Real-time applications

8. npm Ecosystem -

npm = Node Package Manager

Node.js has access to a huge ecosystem of reusable packages through **npm (Node Package Manager)**.

It is used to:

- Install packages
- Manage dependencies
- Run scripts
- Manage project configuration

Run on terminal: ***npm -v***

You should also get a version number.

REPL in Node.js -

REPL = Read-Eval-Print Loop

Node.js provides a built-in interactive JavaScript environment called the **Node.js REPL**.
It allows us to execute JavaScript code directly from the terminal without creating a `.js` file.

Starting REPL -

Open the terminal and type:

node

You will see something similar to:

>

Now you can write JavaScript:

> 10 + 20
30

> "Hello".toUpperCase()
'HELLO'

> let x = 100
undefined

> x + 50
150

R — Read
Reads the JavaScript expression entered by the user.

E — Eval
Evaluates/executes the expression.

P — Print
Prints the result.

L — Loop
Waiting for the next command.

REPL is useful for:

- Learning JavaScript
- Testing small pieces of code
- Debugging
- Experimenting with Node.js APIs
- Quickly checking syntax or expressions

Example -

```
> const name = "node"
undefined
> name
'node'
> name.length
4
```

Exit REPL -

You can exit using: **.exit** or press: ***Ctrl + C*** twice.

Global Objects -

Global objects are objects/functions/values that are available throughout a Node.js application without explicitly importing them.

Example 1:

console.log("Hello");

We don't need to import the `console` first.

Example 2:

setTimeout(() => {

console.log("Hello");

}, 2000);

`setTimeout()` can be used directly.

Example 3:

console.log(process.version);

The `process` object provides information and control over the current Node.js process. This displays the Node.js version.

Understanding the JavaScript Event Loop

Call Stack • Web APIs • Callback Queue • Microtask Queue • The Event Loop

> 👋 **Before we start**
>
> This is one of the most misunderstood topics in JavaScript — and also one of the most rewarding to truly understand. Once the Event Loop "clicks," async code stops feeling like magic and starts feeling like simple, predictable machinery. Take it one section at a time, run the examples yourself, and you'll get there. Let's go!

1. Core Concepts: The Call Stack & the Global Execution Context

1.1 What is the Global Execution Context (GEC)?

The moment a JavaScript file starts running, the JS engine creates a special container called the **Global Execution Context (GEC)**. Think of it as the "main stage" of your program — it is created first, and it is destroyed last. Every other piece of code runs *inside* it, directly or indirectly.

1.2 What is the Call Stack?

The **Call Stack** is a simple data structure that keeps track of *where the program currently is*. It works on a **LIFO** principle — *Last In, First Out*. Whenever a function is called, JavaScript pushes a new frame onto the stack. When that function finishes (returns), its frame is popped off.

> 💡 **Mental Model**
>
> Imagine a stack of plates. The GEC is the table the stack sits on. Every function call adds a plate on top; every function that finishes removes the plate from the top. You can only ever look at (execute) the very top plate.

1.3 Step-by-Step Trace: A Simple Synchronous Example

Let's trace exactly what the JS engine does with this code:

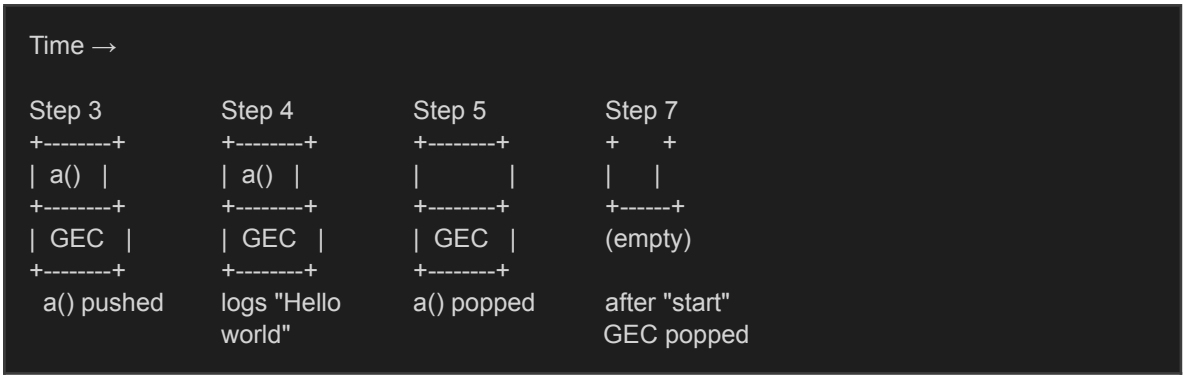
```
function a() {
  console.log("Hello world");
}

a();
console.log("start");
```

Execution trace

1. JS engine starts the file → a Global Execution Context (GEC) is created and pushed onto the Call Stack. Stack: [GEC]
2. Function a() is declared (hoisted) but not run yet — nothing happens on the stack for a declaration.
3. Line a() executes → a new Function Execution Context for a() is created and pushed on top of the GEC. Stack: [GEC, a()]
4. Inside a(), console.log("Hello world") runs → "Hello world" is printed immediately.
5. a() has nothing left to do → its execution context is popped off the stack. Stack: [GEC]
6. Back in the GEC, console.log("start") runs → "start" is printed.
7. The file has no more code → the GEC itself is popped off. Stack: [ ] (empty).

Visualizing the stack over time



// Output:

Hello world
start

> ⚠️ **Common Misconception**
>
> “Functions run in the order they are written.” Not quite — functions run in the order they are CALLED and INVOKED on the stack, which is why a() (called first) fully finishes before console.log("start") even though both lines look sequential. Order comes from the stack, not just the source order.

2. Web APIs / Browser APIs — Why JavaScript Needs Help

2.1 JavaScript itself is single-threaded

Here's the key fact that makes the Event Loop necessary: **the JavaScript engine (like V8) only has ONE Call Stack and can only do ONE thing at a time.** It has *no built-in ability* to "wait" for something (like a timer or a network request) without freezing the entire program.

2.2 So who handles waiting?

JavaScript engines don't run in a vacuum — they run **inside an environment**, such as a **browser** (Chrome, Firefox) or **Node.js**. These environments provide extra capabilities that plain JS does not have, called **Web APIs** in the browser (Node has its own equivalent APIs, e.g. libuv-backed timers and I/O). These include:

- **setTimeout / setInterval** — timers that count down in the background
- **fetch / XMLHttpRequest** — network requests handled outside the JS thread
- **DOM Events** — click, scroll, keypress listeners
- **Geolocation, localStorage, requestAnimationFrame**, and more

> 🔑 **Key Idea**
>
> The browser (or Node) essentially says: "Don't worry, JS — I'll handle the waiting for you. You keep running other code, and I'll let you know the moment the timer finishes / the data arrives." This is what makes JavaScript feel asynchronous even though the engine itself is single-threaded.

2.3 Step-by-Step Trace: Introducing setTimeout

```
console.log("start");

setTimeout(() => {
  console.log("Async Task");
}, 2000);

console.log("end");
```

Execution trace

1. GEC is created and pushed onto the Call Stack.
2. console.log("start") runs → "start" printed immediately.

3. setTimeout(...) is called. JS does NOT run the timer itself — it hands the callback function and the 2000ms delay off to the Web API environment (the browser), and immediately continues to the next line. The setTimeout call itself is popped off the stack right away.
4. console.log("end") runs → "end" printed immediately.
5. The main script finishes → GEC is popped → Call Stack is empty.
6. Meanwhile, the browser's internal timer counts down in the background (this does NOT block the stack).
7. After ~2000ms, the timer completes. The callback () => console.log("Async Task") is moved into the Callback Queue (Task Queue), waiting for its turn.
8. The Event Loop (explained in Section 4) checks: “Is the Call Stack empty?” Yes → it pushes the callback from the queue onto the stack to run it.
9. "Async Task" is finally printed — roughly 2 seconds after the script started, but AFTER "start" and "end".

// Output:

start

end

Async Task (≈ 2 seconds later)

> ⚠️ **Common Misconception**
>
> “setTimeout(fn, 2000) guarantees fn runs at exactly 2000ms.” Not true — 2000ms is the MINIMUM delay before the callback is eligible to move to the queue. If the Call Stack is busy with other code when the timer finishes, the callback has to wait its turn. The delay is a floor, not a promise.

3. Callback Queue vs. Microtask Queue

Once a Web API finishes its job (timer ends, data arrives, etc.), it doesn't throw its callback directly onto the Call Stack. Instead, it places the callback into a *waiting line* — and JavaScript actually has **two** such waiting lines, with different priority.

3.1 The Callback Queue (a.k.a. Macrotask / Task Queue)

Holds callbacks from:

- setTimeout / setInterval
- DOM events (click, keydown, etc.)
- setImmediate (Node.js)
- I/O callbacks in Node.js

3.2 The Microtask Queue (Priority Queue)

Holds callbacks from:

- **Promise**.then() / .catch() / .finally()
- **async/await** (an awaited call resumes as a microtask)
- queueMicrotask()
- **MutationObserver** callbacks

> 🏆 **The Golden Rule**
>
> The Microtask Queue always has priority over the Callback (Macrotask) Queue. After every single task finishes running on the Call Stack, the Event Loop must FULLY drain the entire Microtask Queue — including any NEW microtasks added along the way — before it is allowed to pick even one task from the Callback Queue.

3.3 Visual Comparison

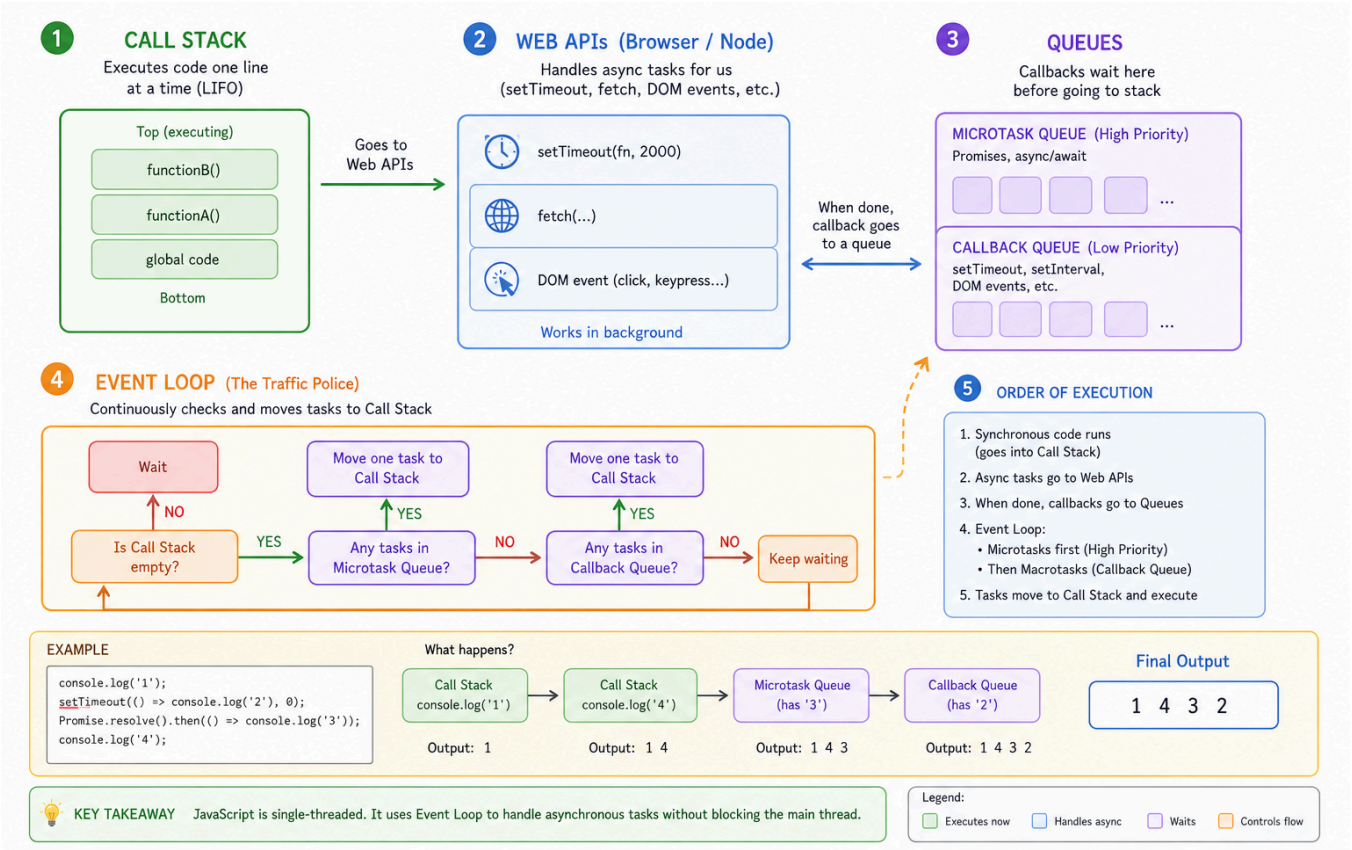
Feature	Callback Queue (Macrotask)	Microtask Queue
Examples	setTimeout, setInterval, DOM events	Promise.then/catch/finally, async/await, queueMicrotask
Priority	Lower — runs after microtasks are empty	Higher — always runs first
How many run per Event Loop tick	Only ONE macrotask is taken per loop cycle	ALL microtasks run, until the queue is fully empty
Nickname	“Task Queue”	“Priority Queue”

4. The Event Loop — The Bridge That Connects Everything

The **Event Loop** is not a queue itself — it's a constantly-running *watchman / traffic controller*. Its ONE job, repeated forever, is beautifully simple:

> 🔁 **The Event Loop's Job (in plain English)**
>
> “Is the Call Stack empty? If yes — first, run every single microtask waiting in the Microtask Queue (fully draining it). Then, take ONE task from the Callback Queue and push it onto the Call Stack. Repeat forever.”

4.1 The full picture



4.2 Why the Event Loop matters

Without the Event Loop, JavaScript would have no way to know *when* it's safe to run a queued callback. It is the mechanism that lets a single-threaded language handle thousands of timers, network requests, and UI events without ever blocking the page — as long as the Call Stack is respected: **never run a queued callback while the stack is still busy.**

5. Full Code Walkthroughs

5.1 Walkthrough A — Basic Async Task

```
console.log("1: Sync start");

setTimeout(() => {
  console.log("2: Timeout callback");
}, 0);

Promise.resolve().then(() => {
  console.log("3: Promise callback");
});

console.log("4: Sync end");
```

Trace

1. "1: Sync start" logs immediately (synchronous, runs on the Call Stack).
2. setTimeout hands its callback to the Web API. Even with a 0ms delay, it still must go through the Callback Queue — it cannot run immediately.
3. Promise.resolve().then(...) schedules its callback into the Microtask Queue.
4. "4: Sync end" logs immediately (still synchronous code running).
5. Call Stack is now empty. Event Loop checks the Microtask Queue FIRST → runs it → "3: Promise callback" logs.
6. The Microtask Queue is now empty. Event Loop moves to the Callback Queue → runs the timeout callback → "2: Timeout callback" logs.

// **Output:** 1: Sync start

// **Output:** 4: Sync end

// **Output:** 3: Promise callback

// **Output:** 2: Timeout callback

> ⚠️ **Common Misconception**
>
> “settimeout(fn, 0) runs instantly, right after the current line.” False — even a 0ms setTimeout must go through the Web API and the Callback Queue, so it ALWAYS runs after the synchronous code AND after all microtasks, no matter how short the delay is.

5.2 Walkthrough B — Priority Battle: setTimeout vs. fetch (Promise)

This example shows exactly why the Microtask Queue is called the “Priority Queue” — even when a Promise resolves later than a timer's Web API work theoretically starts, the microtask still cuts in line ahead of the queued macrotask.

```
console.log("A: script start");

setTimeout(() => {
  console.log("B: setTimeout (macrotask)");
}, 0);

fetch("https://example.com/data")
  .then(() => console.log("C: fetch (microtask)"));

Promise.resolve().then(() => console.log("D: promise (microtask)"));

console.log("E: script end");
```

Trace

1. "A: script start" logs — synchronous.
2. setTimeout registers its callback with the Web API and returns immediately (callback destined for the Callback / Macrotask Queue once the timer elapses).
3. fetch() kicks off a network request via the Web API (this takes real time) and immediately returns a pending Promise; .then() registers a microtask to run once that promise resolves.
4. Promise.resolve().then(...) is already resolved, so its callback is queued into the Microtask Queue right away.
5. "E: script end" logs — synchronous code finishes, Call Stack empties.
6. Event Loop drains the Microtask Queue completely before touching the macrotask queue → "D: promise (microtask)" logs first (it was ready immediately).
7. The 0ms setTimeout callback is likely ready in the Callback Queue around this time too, but the Event Loop must fully empty the Microtask Queue on every pass before taking even one macrotask → "B: setTimeout (macrotask)" logs next (assuming the network hasn't responded yet).
8. Once the network response for fetch() actually arrives (this could be milliseconds or longer, and is NOT guaranteed to beat the timer), its microtask joins the Microtask Queue and is drained before any further macrotasks → "C: fetch (microtask)" logs.

// **Output:** A: script start
// **Output:** E: script end
// **Output:** D: promise (microtask)
// **Output:** B: setTimeout (macrotask) <-- typical order, network permitting
// **Output:** C: fetch (microtask) <-- arrives whenever the network responds

📌 Why does D beat B, but C's timing can vary?

D (Promise.resolve()) is already resolved the instant it's created — so its microtask is ready to run on the very next Event Loop check, ahead of ANY macrotask. B (setTimeout) is a macrotask, so it must wait for the Microtask Queue to be fully empty first. C (fetch) depends on an actual network round-trip — its microtask can only be queued once the response arrives, which is often (but not always) slower than a 0ms timer. The rule that never changes: whenever the microtask queue has something in it, it is drained completely before the next macrotask runs.

Quick-Reference Summary

- **Call Stack** = single-threaded, LIFO, runs synchronous code right now.
- **Web APIs** (browser/Node) = handle waiting (timers, network, DOM) OUTSIDE the JS thread.
- **Callback / Macrotask Queue** = holds setTimeout, DOM event, I/O callbacks. One task per Event Loop cycle.
- **Microtask Queue** = holds Promise/async-await callbacks. ALWAYS fully drained before the next macrotask.
- **Event Loop** = the bridge: “Is the stack empty? Drain microtasks. Then take one macrotask.” Repeat forever.

🎉 You made it!

If this is your first real pass at the Event Loop, don't worry if it takes a couple of re-reads and a few `console.log` experiments in your own browser console to fully click. Every professional JS developer had this exact same “wait, WHY did that print in that order?!” moment at some point. Trace the examples by hand, predict the output before running the code, and you'll master this faster than you think. Happy coding!